Capstone Technical Report

Rafael Lima Araujo Rego, Bram Verlaat, Janniek Graveland,
Tobias van der Meer, Jorrit Paques

January 2026

1 Introduction

In this project, we study a groundwater flow inverse problem on a two-dimensional grid. The unknown quantity is the (log-)permeability field, and the available information is the hydraulic head field produced by a steady-state PDE. Solving the PDE once is not very expensive, but inverse methods typically need to solve it many times, which becomes costly in practice. The inverse problem is also difficult because different permeability fields can produce very similar head fields.

Our goal is to use machine learning to learn this inverse mapping directly. We treat the task as an image-to-image regression problem: the input is the permeability field on a grid, and the output is the corresponding head field on the same grid. Most experiments use a 60×60 resolution, but we also consider 64×64 grids to explore deeper architectures with additional downsampling layers. This allows us to test whether extra depth improves performance when the spatial resolution permits further compression.

We experiment with several model classes. In addition to a U-Net, we test different CNN architectures and fully connected baselines. These alternatives help us understand which architectural choices matter most for this inverse problem, and what trade-offs they imply in practice. We compare the models in terms of predictive performance as well as computational cost. In particular, we report training time, inference time, and the time required to generate a full ensemble of predictions, which is the relevant quantity for ensemble-based inversion workflows.

The remainder of this report describes the dataset and training setup, the model architectures, and the evaluation metrics used to assess prediction quality. Finally we will also return to the requirements to discuss if and how we met these. All code is available in our GitHub repository; the README explains how to run the data generation, training, and evaluation scripts.

2 Datasets and Splits

The datasets are generated using `dataset_gen.py`, which calls functions provided by our supervisor. Each sample consists of a hydraulic conductivity field $k(x, y)$ and the corresponding hydraulic head field $h(x, y)$.

Initially, all data were generated on a 60×60 grid. Later in the project, we also generated data on a 64×64 grid, since it is divisible by powers of two and therefore better suited for deeper CNN and U-Net architectures. The 60×60 grid is used for most experiments, and the 64×64 grid is used only for the final CNN and U-Net models. At the end of the project, our supervisor asked us to also

consider a 64×64 setting, so we did not re-run every model at this resolution. Instead, we evaluate the 64×64 data only with the two best models from the 60×60 experiments.

Final model evaluation is performed using a separate validation dataset, generated independently from the training and test sets. The size of the training set is 5000, the size of the validation set is 2000 and the size of the test set is also 2000.

3 Model Architectures and training

Below we describe the model architectures we tested and the rationale behind each design choice.

3.1 Baseline model: CNN

3.1.1 High-level idea

The baseline CNN is not a generic feed-forward convolutional stack. Instead, it is structured as a stack of multiple smaller models:

1. Two fully connected mappings produce an initial estimate of the head field.
2. The estimate is then refined for four iterations using a small convolutional block that takes both the input field and the current head estimate as input.

This design is motivated by the structure of typical Darcy solutions: they often contain a strong global component (e.g. an overall slope or large-scale trend) together with local corrections induced by spatial heterogeneity in k.

3.1.2 Fully connected baseline (fc1)

At first, this is our most basic CNN, which is a fully connected model, without convolutional layers. The results are surprisingly smooth, which means that convolutional layers are not necessary to obtain smooth predictions. The MAE is 3.073.

```
class Model(nn.Module):

    def __init__(self):
        super().__init__()
        self.relu = nn.ReLU()
        self.fc1 = nn.Linear(3600, 512)
        self.fc2 = nn.Linear(512, 512)
        self.fc3 = nn.Linear(512, 512)
        self.fc4 = nn.Linear(512, 512)
        self.fc5 = nn.Linear(512, 3600)

    def forward(self, x):
        x = x.reshape((-1, 3600))
        x = self.relu(self.fc1(x))
        x = self.relu(self.fc2(x))
        x = self.relu(self.fc3(x))
```

```
        x = self.relu(self.fc4(x))
        x = self.fc5(x)
        return x.reshape((-1, 60, 60))
```

3.1.3 CNN12c-model

This model uses four blocks and double fully connected layers. The idea is that the first double fully connected layers compute an initial estimate of the hydraulic head h, and the blocks then iteratively refine this estimate. Each block consists of four convolutional layers, whose inputs have four channels: the raw input (conductivity), the most recent estimate of h, and two additional channels that use the same data but are first passed through double fully connected layers to help capture the global structure of the h-field. Double fully connected layers are used instead of single ones to reduce the total number of parameters by employing a small number of hidden units. In this model, the architecture is a modified version of cnn12, where more hidden layers are placed at the beginning of the network and fewer at the end, while keeping the number of parameters similar. This design aims to reduce noise in the results, leading to an MAE of 1.918.

```
class Block2(nn.Module):
    def __init__(self, n_hidden=144):
        super().__init__()
        self.relu = nn.ReLU()
        self.fc1 = nn.Linear(3600, n_hidden)
        self.fc2 = nn.Linear(n_hidden, 3600)
        self.fc3 = nn.Linear(3600, n_hidden)
        self.fc4 = nn.Linear(n_hidden, 3600)
        self.prep1 = nn.Sequential(self.fc1, self.relu, self.fc2, self.relu)
        self.prep2 = nn.Sequential(self.fc3, self.relu, self.fc4, self.relu)
        self.conv1 = nn.Conv2d(4, 8, 9, padding='same', padding_mode='zeros')
        self.conv2 = nn.Conv2d(8, 8, 9, padding='same', padding_mode='zeros')
        self.conv3 = nn.Conv2d(8, 8, 9, padding='same', padding_mode='zeros')
        self.conv4 = nn.Conv2d(8, 1, 9, padding='same', padding_mode='zeros')

    def forward(self, x, hr):
        z = torch.empty((x.shape[0], 4, 60, 60), device=x.device)
        z[:, 0, :, :] = x.view(-1, 60, 60)
        z[:, 1, :, :] = self.prep1(x.view(-1, 3600)).view(-1, 60, 60)
        z[:, 2, :, :] = hr.view(-1, 60, 60)
        z[:, 3, :, :] = self.prep2(hr.view(-1, 3600)).view(-1, 60, 60)

        r = self.relu(self.conv1(z))
        r = self.relu(self.conv2(r))
        r = self.relu(self.conv3(r))
        r = self.conv4(r)
        return r


class Model(nn.Module):
    def __init__(self):
        super().__init__()
        self.relu = nn.ReLU()
```

```
        self.fc1 = nn.Linear(3600, 128)
        self.fc2 = nn.Linear(128, 3600)
        self.block_1 = Block2(n_hidden=128)
        self.block_2 = Block2(n_hidden=64)
        self.block_3 = Block2(n_hidden=32)
        self.block_4 = Block2(n_hidden=32)

    def forward(self, x):
        h = self.relu(self.fc1(x.reshape((-1, 3600))))
        h = self.relu(self.fc2(h))
        h = h.reshape((-1, 1, 60, 60))
        h = h - self.block_1(x, h)
        h = h - self.block_2(x, h)
        h = h - self.block_3(x, h)
        h = h - self.block_4(x, h)
        return h.reshape((-1, 60, 60))
```

3.1.4 CNN16-model

This model has 12 convolutional layers together with three residual connections. The MAE is 3.956.

```
class Model(nn.Module):

    def __init__(self):
        super().__init__()
        self.relu = nn.ReLU()
        self.conv1 = nn.Conv2d(1, 8, 11, padding='same', padding_mode='zeros')
        self.conv2 = nn.Conv2d(8, 16, 11, padding='same', padding_mode='zeros')

        self.conv3 = nn.Conv2d(16, 16, 19, padding='same', padding_mode='zeros')
        self.conv4 = nn.Conv2d(16, 16, 19, padding='same', padding_mode='reflect')
        self.conv5 = nn.Conv2d(16, 16, 19, padding='same', padding_mode='reflect')

        self.conv6 = nn.Conv2d(16, 16, 13, padding='same', padding_mode='zeros')
        self.conv7 = nn.Conv2d(16, 16, 13, padding='same', padding_mode='reflect')
        self.conv8 = nn.Conv2d(16, 16, 13, padding='same', padding_mode='reflect')

        self.conv9 = nn.Conv2d(16, 16, 9, padding='same', padding_mode='zeros')
        self.conv10 = nn.Conv2d(16, 16, 9, padding='same', padding_mode='zeros')
        self.conv11 = nn.Conv2d(16, 16, 9, padding='same', padding_mode='zeros')

        self.conv12 = nn.Conv2d(16, 1, 7, padding='same', padding_mode='reflect')

    def forward(self, x):
        h1 = self.relu(self.conv1(x))
        h1 = self.relu(self.conv2(h1))

        h2 = self.relu(self.conv3(h1))
        h2 = self.relu(self.conv4(h2))
        h2 = self.relu(self.conv5(h2)) + h1
```

```
        h3 = self.relu(self.conv6(h2))
        h3 = self.relu(self.conv7(h3))
        h3 = self.relu(self.conv8(h3)) + h2

        h4 = self.relu(self.conv9(h3))
        h4 = self.relu(self.conv10(h4))
        h4 = self.relu(self.conv11(h4)) + h3

        h5 = self.conv12(h4)
        return h5.view(-1, 60, 60)
```

3.1.5 CNN with fully connected head (cnn_fc)

During one of our meetings, we considered adding fully connected layers after a convolutional encoder. The best-performing CNN is this one: it starts with a small convolutional stack (similar to a U-Net encoder) and ends with three fully connected layers. It achieves an MAE of 1.321 and produces smooth outputs with relatively few outliers. When we increased the input grid from 60×60 to 64×64, performance became slightly worse (MAE 1.564), suggesting that the higher-resolution setting is a harder prediction task for this model.

```
class Model(nn.Module):

    def __init__(self):
        super().__init__()
        self.relu = nn.ReLU()
        self.conv1 = nn.Conv2d(1, 16, 5, padding='same', padding_mode='reflect')
        self.conv2 = nn.Conv2d(16, 16, 7, padding='same', padding_mode='zeros')
        self.pool1 = nn.MaxPool2d(2)
        self.conv3 = nn.Conv2d(16, 32, 5, padding=3, padding_mode='zeros')
        self.conv4 = nn.Conv2d(32, 32, 5, padding='same', padding_mode='zeros')
        self.pool2 = nn.MaxPool2d(2)
        self.conv5 = nn.Conv2d(32, 64, 5, padding='same', padding_mode='zeros')
        self.conv6 = nn.Conv2d(64, 64, 5, padding='same', padding_mode='zeros')
        self.pool3 = nn.MaxPool2d(2)
        self.conv7 = nn.Conv2d(64, 64, 5, padding='same', padding_mode='zeros')
        self.conv8 = nn.Conv2d(64, 64, 3, padding='same', padding_mode='zeros')

        self.fc1 = nn.Linear(4096, 2048)
        self.fc2 = nn.Linear(2048, 2048)
        self.fc3 = nn.Linear(2048, 3600)



    def forward(self, x):
        h = self.relu(self.conv1(x))
        h = self.relu(self.conv2(h))
        h = self.pool1(h)

        h = self.relu(self.conv3(h))
```

```
h = self.relu(self.conv4(h))
h = self.pool2(h)

h = self.relu(self.conv5(h))
h = self.relu(self.conv6(h))
h = self.pool3(h)

h = self.relu(self.conv7(h))
h = self.relu(self.conv8(h)).view(-1, 4096)

h = self.relu(self.fc1(h))
h = self.relu(self.fc2(h))
h = self.fc3(h)

return h.reshape((-1, 60, 60))
```

3.2 Upgrade model: U-Net

3.2.1 Why a standard U-Net fits this task

A standard U-Net is an encoder–decoder convolutional network designed for image-to-image prediction. It takes a grid-based input field and produces a grid-based output field at the same spatial resolution (64×64 or 60×60), which matches our setting where both the input and the target should be the same size.

On the encoder side, each level applies a small convolutional feature extractor (two 3×3 convolutions with ReLU) and then a 2×2 max-pooling to halve the resolution. The number of channels increases as we go deeper (often by factors of two) so that the model can store richer features while the receptive field (the region of the input that influences one feature value or a pixel) grows, which helps it capture larger-scale patterns at lower resolution.

At the bottleneck, the network works at the lowest spatial resolution and with the highest channel count. This part acts like a compressed summary of the whole domain, which is appropriate here because the head field is shaped by global constraints (where the general flow is going and boundary conditions), not only by local variations in conductivity.

On the decoder side, the network upsamples step-by-step back to the original resolution while applying convolution blocks at each scale. This is useful because our target field needs to be predicted everywhere on the grid, and the upsampling path lets the model turn the global, low-resolution representation into a full-resolution field in a controlled way.

A key feature is the set of skip connections: at each scale, the decoder receives the corresponding encoder feature map through concatenation (same size, e.g. an 8×8 encoder map concatenated with an 8×8 decoder map). This matters for our task because fine spatial details in the output are strongly linked to local heterogeneity in the input, and skip connections preserve this high-resolution information that would otherwise be weakened by repeated pooling.

Finally, the network uses a small output projection (often a 1×1 convolution) to map the last feature map to the required output channel(s). This keeps the architecture flexible and makes the final layer a simple "readout" of the spatial features the U-Net has learned.

For PDE solution fields, local structure matters (heterogeneous k), but so does global structure (overall pressure gradient). Therefore the U-Net seems to be suited for this problem. The models below are all variants of this same U-Net explained above with only one major Global Context addition in one of them, which was later removed.

3.2.2 U-Net (first version)

Our first U-Net was a shallow encoder–decoder model for the 60×60 setting. With 60×60 inputs we limited the depth to two pooling steps ($60 \to 30 \to 15$) to avoid very small and uneven bottleneck sizes. This limited how much global context the bottleneck could learn. In practice, this first U-Net produced reasonable but suboptimal predictions, and its performance was worse than our best early CNN baselines.

To address this, we added an explicit global context mechanism at the bottleneck. The idea is taken from work on feature recalibration and context modeling in computer vision, especially Squeeze-and-Excitation Networks [1] and GCNet [2]. The core message of these papers is that standard convolutions are very good at local pattern extraction, but they can be inefficient at capturing domain-wide dependencies. For our problem this matters because the head field is not only shaped by local heterogeneity in k, but also by global effects such as the overall hydraulic gradient and boundary-driven flow direction.

Concretely, we inserted a small global injection block at the 15×15 bottleneck. First, we apply global average pooling to the bottleneck feature map, which compresses each channel to a single number. This creates a compact summary of the entire domain. Second, we pass this pooled vector through a small MLP (two linear layers with a ReLU) to learn how each channel should be adjusted based on the global state of the sample. Finally, we reshape the MLP output back to $(C, 1, 1)$ and add it to the bottleneck feature map through broadcasting. In plain terms, this gives the network a cheap way to look at the whole domain and then adjust its bottleneck representation in a globally consistent direction before decoding back to full resolution.

The rest of the architecture is a standard U-Net: we upsample step-by-step and use skip connections to bring back high-resolution encoder features, so that the decoder can combine global structure with local detail (same as before).

We decided to investigate global context mechanisms after observing that CNN models with an explicit global component (fully connected layers) performed better than models relying only on local convolutions.

```
class Model(nn.Module):

def __init__(self, base_ch: int = 32, enforce_dirichlet_row0: bool = False):
    super().__init__()

    self.enforce_dirichlet_row0 = enforce_dirichlet_row0
    in_ch = 1

    self.enc1 = ConvBlock(in_ch, base_ch)
    self.pool1 = nn.MaxPool2d(2)

    self.enc2 = ConvBlock(base_ch, 2 * base_ch)
```

```
        self.pool2 = nn.MaxPool2d(2)

        self.center = ConvBlock(2 * base_ch, 4 * base_ch)

        self.global_pool = nn.AdaptiveAvgPool2d(1)

        feature_ch = 4 * base_ch

        self.global_dense = nn.Sequential(
            nn.Flatten(),
            nn.Linear(feature_ch, feature_ch),
            nn.ReLU(inplace=True),
            nn.Linear(feature_ch, feature_ch),
            nn.Unflatten(1, (feature_ch, 1, 1))
        )

        self.up2 = nn.Upsample(scale_factor=2, mode="bilinear", align_corners=False)
        self.dec2 = ConvBlock(4 * base_ch + 2 * base_ch, 2 * base_ch)

        self.up1 = nn.Upsample(scale_factor=2, mode="bilinear", align_corners=False)
        self.dec1 = ConvBlock(2 * base_ch + base_ch, base_ch)

        self.out = nn.Conv2d(base_ch, 1, kernel_size=1)

        self.dirichlet_row0_value = (100.0 - 145.3243) / 35.5957

    def forward(self, x: torch.Tensor) -> torch.Tensor:

        x1 = self.enc1(x)
        x2 = self.enc2(self.pool1(x1))

        x_center = self.pool2(x2)
        x_center = self.center(x_center)

        global_feat = self.global_pool(x_center)
        global_feat = self.global_dense(global_feat)
        x_center = x_center + global_feat

        d2 = self.up2(x_center)
        d2 = torch.cat([d2, x2], dim=1)
        d2 = self.dec2(d2)

        d1 = self.up1(d2)
        d1 = torch.cat([d1, x1], dim=1)
        d1 = self.dec1(d1)

        out = self.out(d1)

        if self.enforce_dirichlet_row0:
```

```
            out[:, :, 0, :] = self.dirichlet_row0_value

        return out
```

3.2.3 U-Net deeper $64x64 \to 8x8$ version

After testing the first U-Net, we suspected that part of the remaining error came from limited depth rather than from the general encoder–decoder design. With only two pooling steps, the bottleneck representation was still relatively high-resolution, which restricts how much long-range information can be mixed before decoding. This was when our supervisor decided to move from the 60x60 grid to a 64x64 grid. We therefore moved to a deeper U-Net setting where an additional pooling level is possible, so the encoder compresses the field further before reconstruction.

In this version, the encoder applies the same ConvBlock pattern as before, but now with three pooling stages. This creates a smaller bottleneck (here $64 \times 64 \to 32 \times 32 \to 16 \times 16 \to 8 \times 8$). Besides this, we have the same architecture as before and we maintained the global context injection.

```
class ConvBlock(nn.Module):
    def __init__(self, in_ch: int, out_ch: int):
        super().__init__()
        self.conv1 = nn.Conv2d(in_ch, out_ch, kernel_size=3, padding=1)
        self.conv2 = nn.Conv2d(out_ch, out_ch, kernel_size=3, padding=1)
        self.relu = nn.ReLU(inplace=True)

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        x = self.relu(self.conv1(x))
        x = self.relu(self.conv2(x))
        return x


class Model(nn.Module):
    """
    Deeper U-Net for 64x64 grid.
    Depth: 3 pools => bottleneck at 8x8
    Includes global injection in bottleneck.
    """

    def __init__(self, base_ch: int = 64, enforce_dirichlet_row0: bool = True):
        super().__init__()
        self.enforce_dirichlet_row0 = enforce_dirichlet_row0

        in_ch = 1

        self.enc1 = ConvBlock(in_ch, base_ch)
        self.pool1 = nn.MaxPool2d(2)

        self.enc2 = ConvBlock(base_ch, 2 * base_ch)
        self.pool2 = nn.MaxPool2d(2)
```

```
        self.enc3 = ConvBlock(2 * base_ch, 4 * base_ch)
        self.pool3 = nn.MaxPool2d(2)

        self.center = ConvBlock(4 * base_ch, 8 * base_ch)

        self.global_pool = nn.AdaptiveAvgPool2d(1)
        feature_ch = 8 * base_ch

        self.global_dense = nn.Sequential(
            nn.Flatten(),
            nn.Linear(feature_ch, feature_ch),
            nn.ReLU(inplace=True),
            nn.Linear(feature_ch, feature_ch),
            nn.Unflatten(1, (feature_ch, 1, 1))
        )

        self.up3 = nn.Upsample(scale_factor=2, mode="bilinear", align_corners=False)
        self.dec3 = ConvBlock(8 * base_ch + 4 * base_ch, 4 * base_ch)

        self.up2 = nn.Upsample(scale_factor=2, mode="bilinear", align_corners=False)
        self.dec2 = ConvBlock(4 * base_ch + 2 * base_ch, 2 * base_ch)

        self.up1 = nn.Upsample(scale_factor=2, mode="bilinear", align_corners=False)
        self.dec1 = ConvBlock(2 * base_ch + base_ch, base_ch)

        self.out = nn.Conv2d(base_ch, 1, kernel_size=1)

        self.dirichlet_row0_value = (100.0 - 145.3243) / 35.5957

    def forward(self, x: torch.Tensor) -> torch.Tensor:

        x1 = self.enc1(x)
```

3.2.4 U-Net deeper $64 \times 64 \to 4 \times 4$ version

This version extends the previous U-Net by adding one extra downsampling level, so the encoder compresses the input to a 4×4 bottleneck instead of 8×8. Apart from this added depth, the architecture follows the same design as the U-Net 8×8 above.

```
class ConvBlock(nn.Module):
    def __init__(self, in_ch: int, out_ch: int):
        super().__init__()
        self.conv1 = nn.Conv2d(in_ch, out_ch, kernel_size=3, padding=1)
        self.conv2 = nn.Conv2d(out_ch, out_ch, kernel_size=3, padding=1)
        self.relu = nn.ReLU(inplace=True)

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        x = self.relu(self.conv1(x))
```

```
        x = self.relu(self.conv2(x))
        return x


class Model(nn.Module):
    """
    U-Net for 64x64 grid with 4 downsampling steps:
    64 -> 32 -> 16 -> 8 -> 4 bottleneck

    Includes 'Global Brain' injection in bottleneck.
    """

    def __init__(self, base_ch: int = 64, enforce_dirichlet_row0: bool = True):
        super().__init__()
        self.enforce_dirichlet_row0 = enforce_dirichlet_row0

        in_ch = 1

        self.enc1 = ConvBlock(in_ch, base_ch)
        self.pool1 = nn.MaxPool2d(2)

        self.enc2 = ConvBlock(base_ch, 2 * base_ch)
        self.pool2 = nn.MaxPool2d(2)

        self.enc3 = ConvBlock(2 * base_ch, 4 * base_ch)
        self.pool3 = nn.MaxPool2d(2)

        self.enc4 = ConvBlock(4 * base_ch, 8 * base_ch)
        self.pool4 = nn.MaxPool2d(2)

        self.center = ConvBlock(8 * base_ch, 16 * base_ch)

        self.global_pool = nn.AdaptiveAvgPool2d(1)
        feature_ch = 16 * base_ch

        self.global_dense = nn.Sequential(
            nn.Flatten(),
            nn.Linear(feature_ch, feature_ch),
            nn.ReLU(inplace=True),
            nn.Linear(feature_ch, feature_ch),
            nn.Unflatten(1, (feature_ch, 1, 1))
        )

        self.up4 = nn.Upsample(scale_factor=2, mode="bilinear", align_corners=False)
        self.dec4 = ConvBlock(16 * base_ch + 8 * base_ch, 8 * base_ch)

        self.up3 = nn.Upsample(scale_factor=2, mode="bilinear", align_corners=False)
        self.dec3 = ConvBlock(8 * base_ch + 4 * base_ch, 4 * base_ch)
```

```
        self.up2 = nn.Upsample(scale_factor=2, mode="bilinear", align_corners=False)
        self.dec2 = ConvBlock(4 * base_ch + 2 * base_ch, 2 * base_ch)

        self.up1 = nn.Upsample(scale_factor=2, mode="bilinear", align_corners=False)
        self.dec1 = ConvBlock(2 * base_ch + base_ch, base_ch)

        self.out = nn.Conv2d(base_ch, 1, kernel_size=1)

        self.dirichlet_row0_value = (100.0 - 145.3243) / 35.5957

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        x1 = self.enc1(x)
        x2 = self.enc2(self.pool1(x1))
        x3 = self.enc3(self.pool2(x2))
        x4 = self.enc4(self.pool3(x3))

        x_center = self.pool4(x4)
        x_center = self.center(x_center)

        global_feat = self.global_pool(x_center)
        global_feat = self.global_dense(global_feat)
        x_center = x_center + global_feat

        d4 = self.up4(x_center)
        d4 = torch.cat([d4, x4], dim=1)
        d4 = self.dec4(d4)

        d3 = self.up3(d4)
        d3 = torch.cat([d3, x3], dim=1)
        d3 = self.dec3(d3)

        d2 = self.up2(d3)
        d2 = torch.cat([d2, x2], dim=1)
        d2 = self.dec2(d2)

        d1 = self.up1(d2)
        d1 = torch.cat([d1, x1], dim=1)
        d1 = self.dec1(d1)

        out = self.out(d1)

        if self.enforce_dirichlet_row0:
            out = out.clone()
            out[:, :, 0, :] = self.dirichlet_row0_value

        return out
```

3.2.5 U-Net deeper $64 \times 64 \to 4 \times 4$ without global context injection

To analyse the effect of the global context mechanism used in the bottleneck of our other U-Net variants, we trained the same deep U-Net architecture but removed the global injection block. The motivation was that, once the network is deep enough and the bottleneck is very small (4×4), each bottleneck feature already aggregates information from a large part of the domain. We therefore wanted to test whether the explicit global context injection still provides additional benefit, or whether the standard encoder–decoder structure is sufficient at this depth.

```
class ConvBlock(nn.Module):
    def __init__(self, in_ch: int, out_ch: int):
        super().__init__()
        self.conv1 = nn.Conv2d(in_ch, out_ch, kernel_size=3, padding=1)
        self.conv2 = nn.Conv2d(out_ch, out_ch, kernel_size=3, padding=1)
        self.relu = nn.ReLU(inplace=True)

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        x = self.relu(self.conv1(x))
        x = self.relu(self.conv2(x))
        return x


class Model(nn.Module):
    """
    U-Net for 64x64 grid with 4 downsampling steps:
    64 -> 32 -> 16 -> 8 -> 4 bottleneck

    NO global injection.
    """

    def __init__(self, base_ch: int = 64, enforce_dirichlet_row0: bool = True):
        super().__init__()
        self.enforce_dirichlet_row0 = enforce_dirichlet_row0

        in_ch = 1


        self.enc1 = ConvBlock(in_ch, base_ch)
        self.pool1 = nn.MaxPool2d(2)

        self.enc2 = ConvBlock(base_ch, 2 * base_ch)
        self.pool2 = nn.MaxPool2d(2)

        self.enc3 = ConvBlock(2 * base_ch, 4 * base_ch)
        self.pool3 = nn.MaxPool2d(2)

        self.enc4 = ConvBlock(4 * base_ch, 8 * base_ch)
        self.pool4 = nn.MaxPool2d(2)

        self.center = ConvBlock(8 * base_ch, 16 * base_ch)
```

```
        self.up4 = nn.Upsample(scale_factor=2, mode="bilinear", align_corners=False)
        self.dec4 = ConvBlock(16 * base_ch + 8 * base_ch, 8 * base_ch)

        self.up3 = nn.Upsample(scale_factor=2, mode="bilinear", align_corners=False)
        self.dec3 = ConvBlock(8 * base_ch + 4 * base_ch, 4 * base_ch)

        self.up2 = nn.Upsample(scale_factor=2, mode="bilinear", align_corners=False)
        self.dec2 = ConvBlock(4 * base_ch + 2 * base_ch, 2 * base_ch)

        self.up1 = nn.Upsample(scale_factor=2, mode="bilinear", align_corners=False)
        self.dec1 = ConvBlock(2 * base_ch + base_ch, base_ch)

        self.out = nn.Conv2d(base_ch, 1, kernel_size=1)

        self.dirichlet_row0_value = (100.0 - 145.3243) / 35.5957

    def forward(self, x: torch.Tensor) -> torch.Tensor:

        x1 = self.enc1(x)
        x2 = self.enc2(self.pool1(x1))
        x3 = self.enc3(self.pool2(x2))
        x4 = self.enc4(self.pool3(x3))

        x_center = self.pool4(x4)
        x_center = self.center(x_center)

        d4 = self.up4(x_center)
        d4 = torch.cat([d4, x4], dim=1)
        d4 = self.dec4(d4)

        d3 = self.up3(d4)
        d3 = torch.cat([d3, x3], dim=1)
        d3 = self.dec3(d3)

        d2 = self.up2(d3)
        d2 = torch.cat([d2, x2], dim=1)
        d2 = self.dec2(d2)

        d1 = self.up1(d2)
        d1 = torch.cat([d1, x1], dim=1)
        d1 = self.dec1(d1)

        out = self.out(d1)

        if self.enforce_dirichlet_row0:
            out = out.clone()
            out[:, :, 0, :] = self.dirichlet_row0_value
        return out
```

3.3 Model training and physics-aware loss attempts

The loss function used for both the U-Net and CNN models is mean squared error (MSE). We used a learning-rate scheduler with a patience window: if the validation loss does not improve for several epochs, the learning rate is reduced. We used a batch size of 16 for all models; increasing the batch size did not improve performance.

We also tested physics-aware training by adding boundary-aware and physics-inspired penalty terms to the loss (i.e., penalizing deviations from the prescribed boundary behavior). In our experiments, these physics-aware variants did not improve results and often reduced predictive accuracy, so we did not use them in the final evaluation. Figure 1 shows a typical prediction obtained with this approach.

Near Neumann-type boundaries, contour lines may bunch up and become parallel because the model has no information about the hydraulic head outside the domain. As a result, predictions near the edges are strongly influenced by the imposed boundary conditions and one-sided gradient approximations, which can produce steep gradients normal to the boundary.

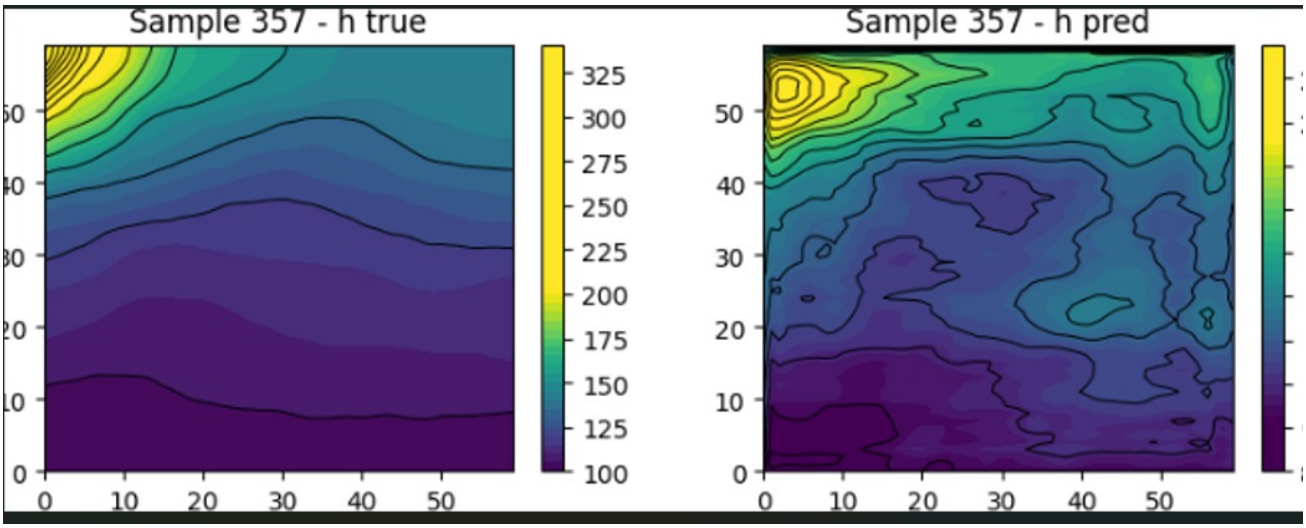

Figure 1: A sample prediction with physical loss function

3.4 Training curves

In figure 2 there is a convergence of the training loss curve after 30 epochs, which means that it is not effective to train it with more epochs as used here. The test loss converges to almost the same rate as the training loss, which means that there is no sign of overfitting.

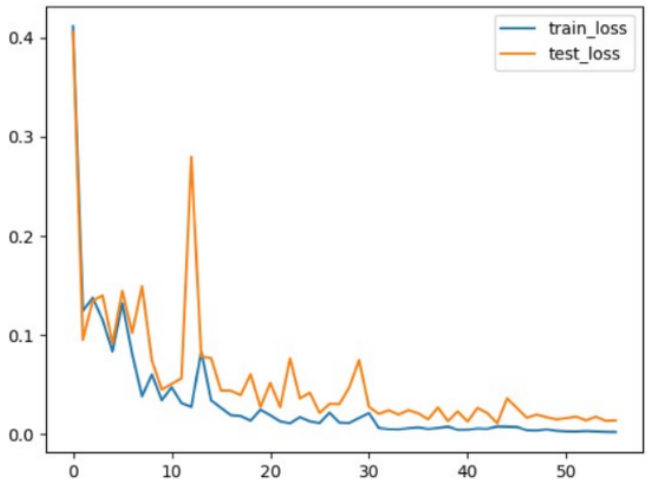

Figure 2: Training curve U-Net44_noglob

Analyzing the curves in figure 3, we again observe a convergence for both the training and test loss, as it takes some more epochs in comparison with the U-Net training curves. Furthermore, there is a bigger difference between the converged train and test loss compared to the curves in figure 2, but it is still not significant enough to call it overfitting.

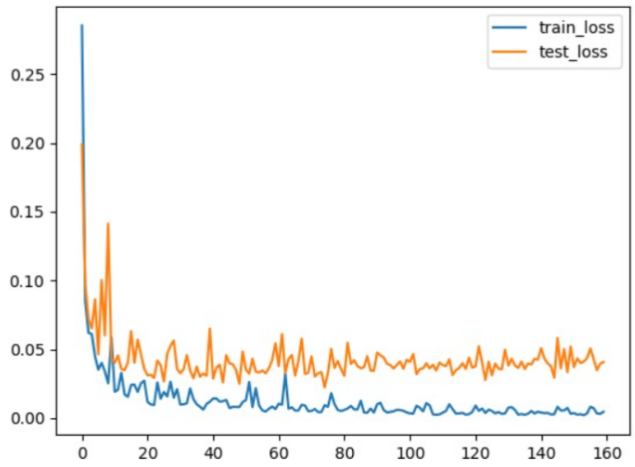

Figure 3: Training curve CNN_fc64

3.5 Performance Validation and Comparison

A core motivation behind the project is to improve the computational time relative to the true calculated field. On the, yet unseen, validation set, the performance of the two best U-nets and four best CNN's is shown below. The best model will be chosen according to the best trade of between inference time and performance.

Table 1: Performance Results Models

model	postfix	input size	#parameters	MAE	avg t/batch infer GPU(ms)	avg t/batch infer CPU(ms)	train time
unet44	-	64	33,476,993	1.428	86.98	4634.93	-
unet44-noglob	-	64	31,377,793	**1.135**	88.96	3560.28	4456.9
unet88	-	64	8,307,073	2.580	70.41	1450.40	5711.6
unet	-	60	2,013,569	1.640	48.46	1816.56	-
cnn_fc64	_b16	64	21,324,272	1.564	6.97	38.62	735.2
cnn_fc	_lr2e-5	60	20,307,968	**1.321**	9.00	32.90	2345.9
cnn12c	_lr6e-5	60	4,695,572	1.918	9.20	349.01	-
cnn16	_test	60	486,657	3.956	75.46	544.01	1769.2
fc1	-	60	4,478,480	3.073	3.98	2.12	-

infer = inference, t/batch = time per batch, avg = average, batch size = 50

The table highlights a clear trade-off between accuracy and computational cost. The U-Net models achieve the lowest MAE, with U-Net 4x4 without globalization performing best, but they are very expensive computationally, especially for CPU inference. In contrast, the CNN_fc models reach comparable accuracy with dramatically lower inference times, making them much more efficient in practice. Smaller CNN and fully connected models are fastest but suffer from significantly worse accuracy, indicating that moderate-sized CNN_fc architectures provide the best balance between performance and efficiency. When comparing the U-Net, the results increase whenever the U-Net becomes deeper. Finally, removing the globalization decreased the error even more and significantly decreased the inference time on the CPU. It also increases the number of parameters as can be seen in the table. The following figures show the true hydraulic head and the predicted hydraulic head from unet44_noglob and cnn_fc.

3.6 Predicted Grids Analyse

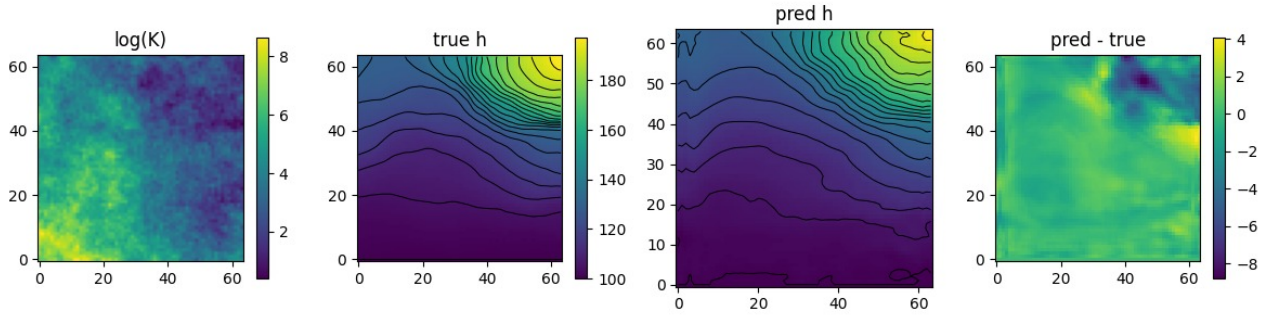

Figure 4: U-Net44_noglob

Looking at the hydraulic head grids predicted by the U-Net 4×4 without the global bottleneck injection, we observe that the predicted hydraulic head field is almost the same as the true hydraulic head field. Only at the bottom, the model has some difficulties with predicting the exact height lines.

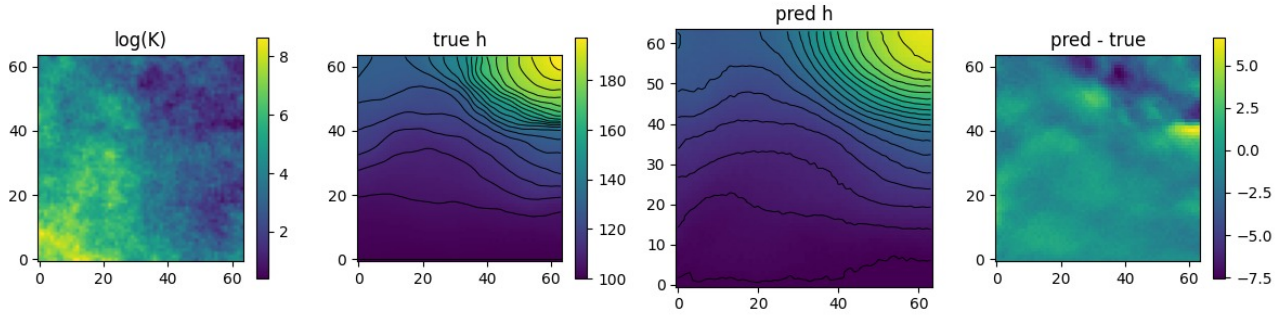

Figure 5: cnn_fc64

Analyzing the plots predicted by the best CNN-model, it is apparent that the lines are a bit more noisy compared to the lines of the best U-Net model. However, the overall prediction is still accurate.

4 Conclusion

4.1 Time Comparison to Physical Field

Model	MAE	Inference Time GPU(ms)	Inference time CPU(ms)
UNet44_noglob	1.135	88.96	3560.28
CNN_fc	1.321	9.00	32.90
Physically calculated field (h-true) 60x60	-	-	4,407.3
Physically calculated field (h-true) 64x64	-	-	5,224.4

In the table above, our best U-Net and the best CNN are shown. From the comparison, the CNN clearly offers the best practical choice. Although the U-Net achieves competitive accuracy its inference time, especially on the CPU, is several orders of magnitude higher, making it unsuitable for fast or large-scale evaluations. The CNN attains comparable MAE while being dramatically faster than both the U-Net and the physically calculated solution, leading us to conclude that the CNN_fc64 provides the best balance between accuracy and computational efficiency and is therefore the recommended model to use. (or the cnn_fc64 if the desired imput is 64×64).

4.2 Cases where the models perform less

In figure 6 the worst 2 samples for the CNN_fc64 are shown.

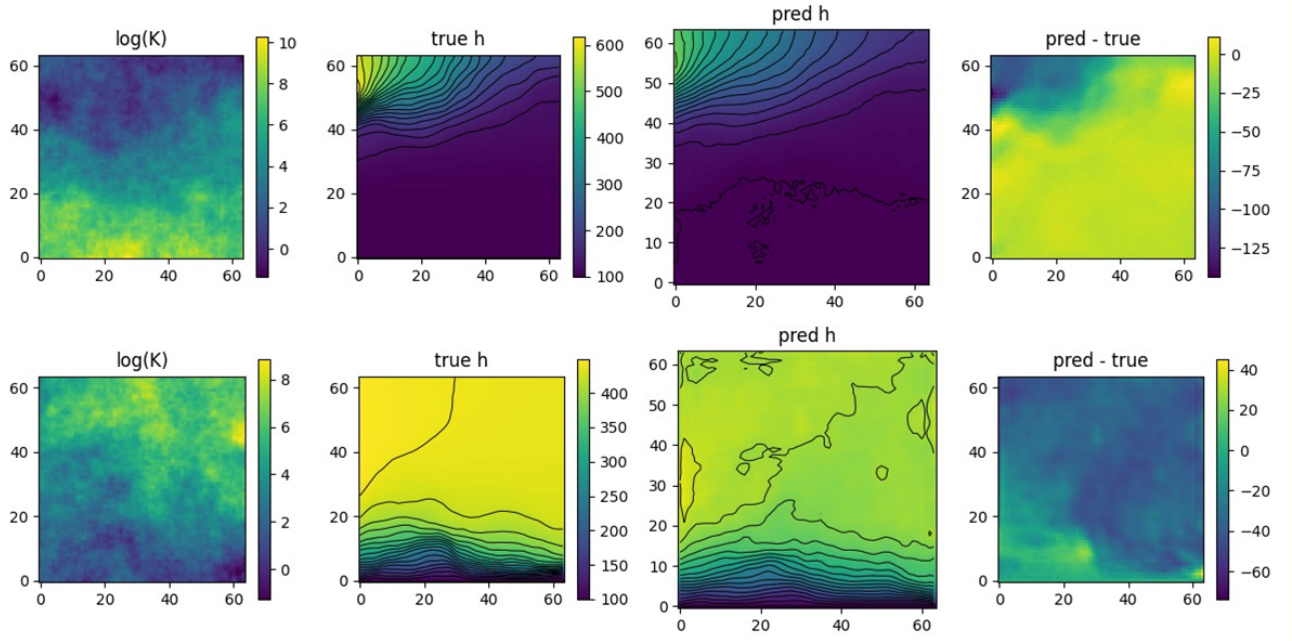

Figure 6: Worst cases CNN

The model seem to not be able to cope with concentrated spots of low hydraulic conductivity values, especially close to the border. this is seen on the left top corner of the upper image, and in the right bottom corner of the lower image.

for the U-NET44_noglob model the worst 2 samples are shown in figure 7.

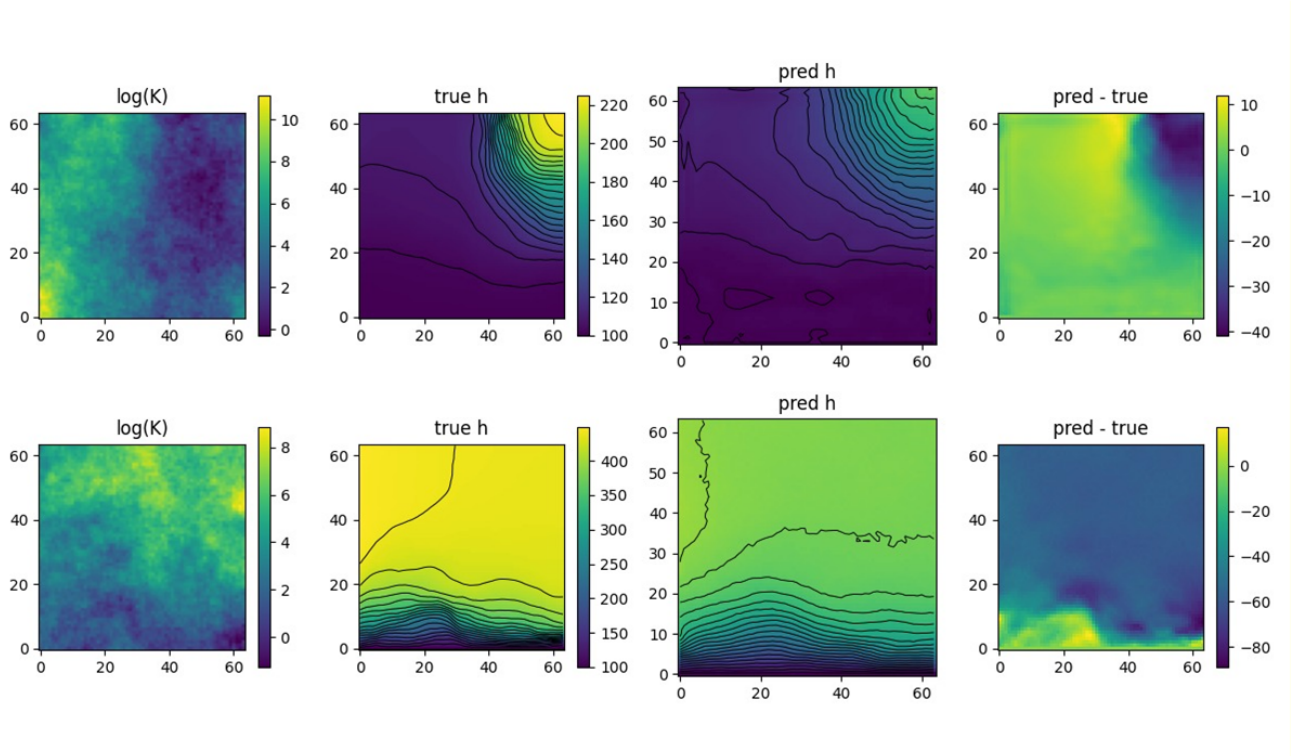

Figure 7: Worst cases U-net

The U-Net seems to have the same trend as the CNN, as one of the samples also seen in the worst cases for the CNN, and the other sample also shows a concentrated low spot.
We tried to quantify what samples would be predicted poorly by taking the minimum value of the conductivity close to the border and we plotted this against the log of the MAE. This plot can be seen in figure 8 (and this plot was generated in performane_analyser.py by setting SHOW_BADNESS_SCORE_PLOT to True.). A clear correlation can be seen here, showing that low conductivities indeed cause difficulties.

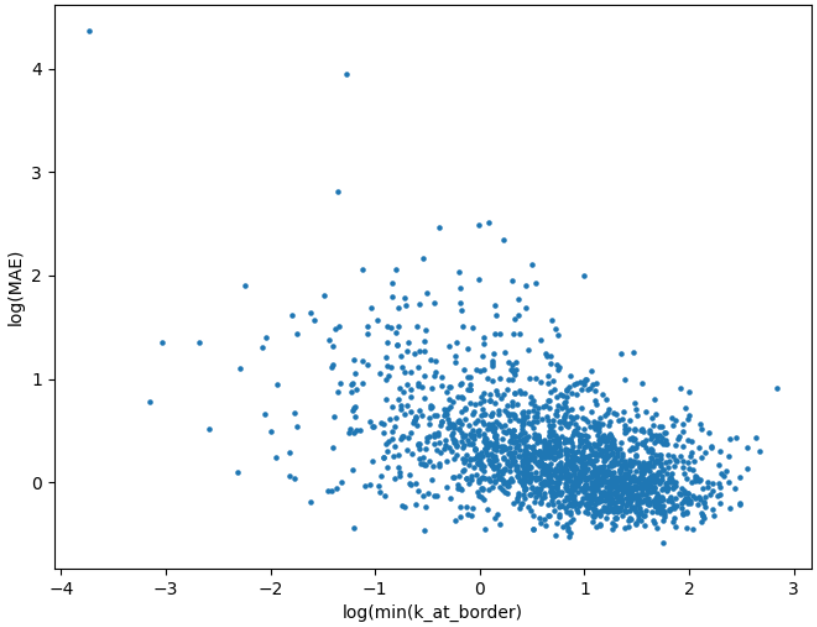

Figure 8: Correlation between minimum value of conductivity and log of MAE for cnn_fc64

5 How we meet the requirements

This section explains how we meet all the requirements.

M1

In `dataset_gen.py`, we import `solve_darcy_flow` and use it to generate the dataset. This function is the Darcy flow solver provided by our supervisor.

M2

In the `models/` folder, each subfolder corresponds to one trained model. The file `model.py` contains the model definition, and the `.pt` files contain the trained weights. CNN models include `cnn` in their name, and U-Net models include `unet` in their name. For details on each model, we refer to this report and the comments inside the corresponding `model.py`.

M3

The file performance_analyser.py is used to fairly estimate the models performance. For this a true test set is used, the model has never seen this set during training. Unfortunately, we named our validation split `test_set`. In this codebase, `test_set` is used for early stopping, while `true_test_set` is used only for final evaluation and is never seen during training. The flags in this file can be changed to show the convergence plots of most models or show examples of inference in addition to the MAE on the `true_test_set`. This file was used to compute the MAE for each model and the inference time.

S1

We put a table with the results of the models in this report. In the table we compare the models based on their MAE.

S2

Model inference time is measured in `performance_analyser.py`. The flag `TEST_ON_GPU` can be set to `True` or `False` to run the timing on GPU or CPU, respectively. The file `darcy_solver_timer.py` measures the runtime of the Darcy solver. The results are reported in a table in this report.

S3

This report explains our codebase and the rationale behind our decisions.

C1

We tested explicit global-information mechanisms in both model families. In the U-Net variants, all models except `unet44_noglob` (since here the depth automatically captured the global structure) use a bottleneck global context injection: global average pooling summarizes the full domain, a small MLP transforms this summary, and the result is added back to the bottleneck features to modulate the decoding.

In the CNN variants, we introduced global mixing through fully connected layers, most clearly in `cnn_fc` (conv encoder + dense layers) and also within models like `cnn12c`. These additions were motivated.

C2

We experimented with physics-aware training by adding physical information to the loss function. In `physics_loss.py`, this loss includes penalties related to the boundary conditions and a PDE-residual term. In our experiments, these additions did not improve performance, so we did not use them in the final models. We also implemented physics-based padding in `cnn13`, but this likewise did not improve results and was not included in the final evaluation.

C3

We trained several deeper CNNs that use only convolutional layers (e.g. `cnn14`, `cnn15`, `cnn16`) to test whether depth alone is enough to capture the global structure of the Darcy solution. In practice, these models did not perform well compared to architectures with an explicit global component.

A likely reason is that plain convolutions are local operators: each layer mixes information only within a small neighborhood. Even though stacking many layers increases the effective receptive field, learning a consistent domain-wide trend (such as the overall hydraulic gradient induced by boundary conditions) can be inefficient and unstable, because the network must propagate global information through many local steps. By contrast, adding a fully connected stage (as in `cnn_fc`) or an explicit global bottleneck mechanism (as in our U-Net variants) gives the model a direct way to condition on the whole domain, which makes global structure easier to represent while convolutions handle local corrections.

References

[1] J. Hu, L. Shen, and G. Sun, "Squeeze-and-Excitation Networks," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2018, pp. 7132–7141. doi:10.1109/CVPR.2018.00745. Available: `https://openaccess.thecvf.com/content_cvpr_2018/html/Hu_Squeeze-and-Excitation_Networks_CVPR_2018_paper.html`

[2] Y. Cao, J. Xu, S. Lin, F. Wei, and H. Hu, "GCNet: Non-local Networks Meet Squeeze-Excitation Networks and Beyond," *arXiv preprint* arXiv:1904.11492, 2019. doi:10.48550/arXiv.1904.11492. Available: `https://arxiv.org/abs/1904.11492`